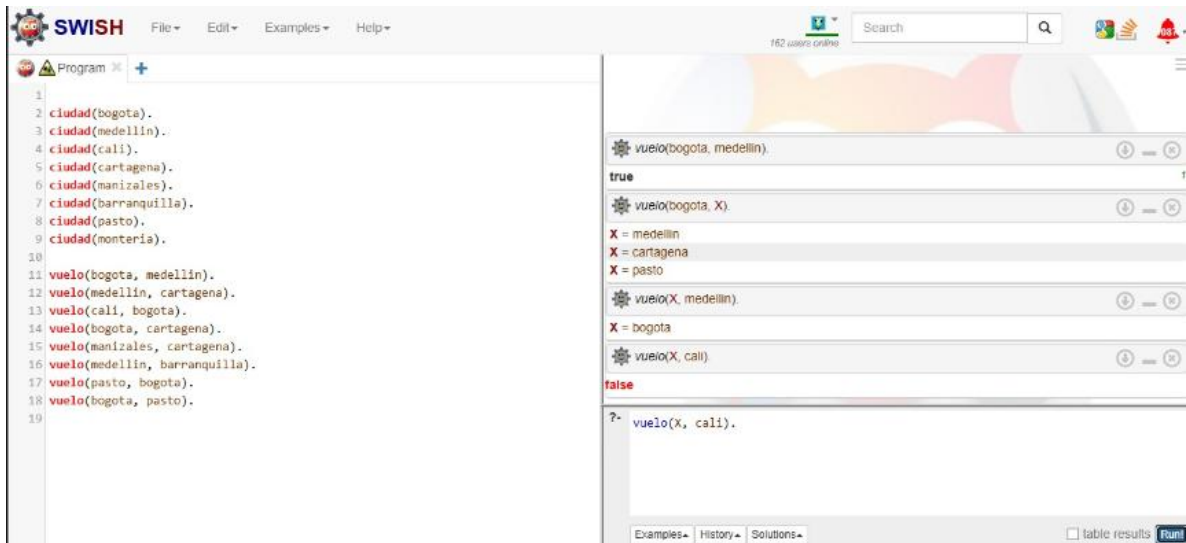


Lab 01 samuel garcia Villamarin IAPP

Ejercicio1



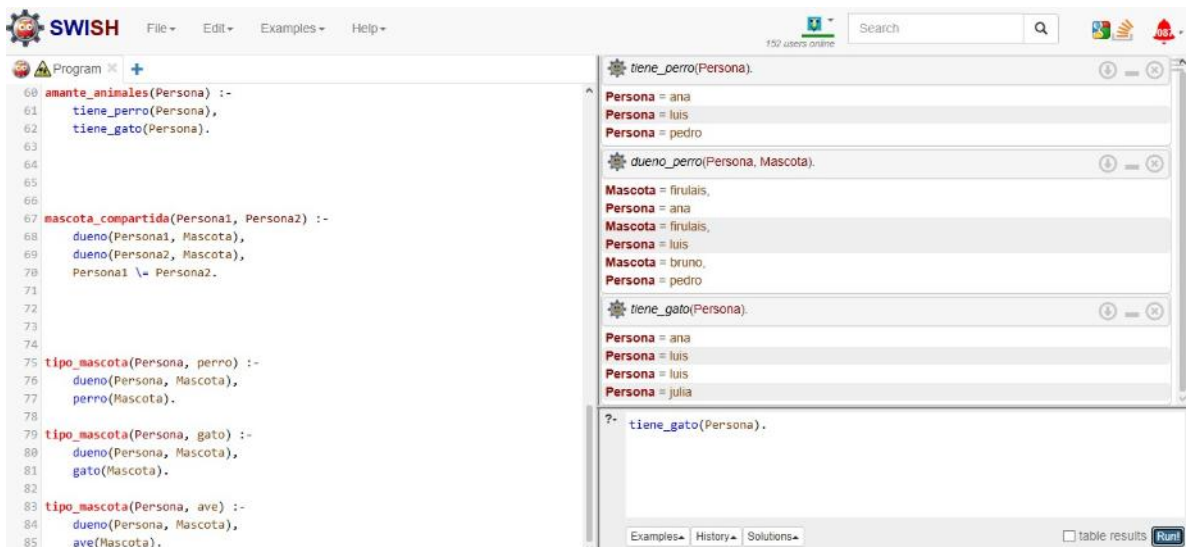
The screenshot shows the SWISH Prolog IDE interface. The left pane contains a Prolog program with the following code:

```
1 ciudad(bogota).
2 ciudad(medellin).
3 ciudad(cali).
4 ciudad(cartagena).
5 ciudad(manizales).
6 ciudad(barranquilla).
7 ciudad(pasto).
8 ciudad(nonteria).
9
10 vuelo(bogota, medellin).
11 vuelo(medellin, cartagena).
12 vuelo(cali, bogota).
13 vuelo(bogota, cartagena).
14 vuelo(manizales, cartagena).
15 vuelo(medellin, barranquilla).
16 vuelo(pasto, bogota).
17 vuelo(bogota, pasto).
18
19
```

The right pane shows the execution results for the query `vuelo(bogota, medellin).` with the following output:

```
true
vuelo(bogota, X).
X = medellin
X = cartagena
X = pasto
vuelo(X, medellin).
X = bogota
vuelo(X, cali).
false
?- vuelo(X, cali).
```

Ejercicio2



The screenshot shows the SWISH Prolog IDE interface. The left pane contains a Prolog program with the following code:

```
60 amante_animales(Persona) :-
61     tiene_perro(Persona),
62     tiene_gato(Persona).
63
64
65
66
67 mascota_compartida(Persona1, Persona2) :-
68     dueno(Persona1, Mascota),
69     dueno(Persona2, Mascota),
70     Persona1 \= Persona2.
71
72
73
74
75 tipo_mascota(Persona, perro) :-
76     dueno(Persona, Mascota),
77     perro(Mascota).
78
79 tipo_mascota(Persona, gato) :-
80     dueno(Persona, Mascota),
81     gato(Mascota).
82
83 tipo_mascota(Persona, ave) :-
84     dueno(Persona, Mascota),
85     ave(Mascota).
```

The right pane shows the execution results for the query `tiene_perro(Persona).` with the following output:

```
tiene_perro(Persona).
Persona = ana
Persona = luis
Persona = pedro
dueno_perro(Persona, Mascota).
Mascota = firulais,
Persona = ana
Mascota = firulais,
Persona = luis
Mascota = bruno,
Persona = pedro
tiene_gato(Persona).
Persona = ana
Persona = luis
Persona = luis
Persona = julia
?- tiene_gato(Persona).
```

SWISH File Edit Examples Help 152 users online Search

Program

```

1 perro(firulaís).
2 perro(bruno).
3 perro(max).
4 gato(misu).
5 gato(luna).
6 gato(chanel).
7 gato(orion).
8 ave(piolin).
9
10 dueño(ana, firulaís).
11 dueño(ana, misu).
12 dueño(luis, luna).
13 dueño(luis, orion).
14 dueño(luis, firulaís).
15 dueño(maria, piolin).
16 dueño(julia, chanel).
17 dueño(pedro, bruno).
18
19
20 tiene_perro(Persona) :-
21     dueño(Persona, Mascota),
22     perro(Mascota).
23
24
25
26 dueño_perro(Persona, Mascota) :-

```

Mascota = misu,
Persona = ana
Mascota = luna,
Persona = luis
Mascota = orion,
Persona = luis
Mascota = chanel,
Persona = julia

tiene_multiples_mascotas(Persona).

Persona = ana
Persona = ana
Persona = luis
Persona = luis
Persona = luis
Persona = luis
Persona = luis
Persona = luis

?- tiene_multiples_mascotas(Persona).

Examples History Solutions ☐ table results Run

SWISH File Edit Examples Help 152 users online Search

Program

```

1 perro(firulaís).
2 perro(bruno).
3 perro(max).
4 gato(misu).
5 gato(luna).
6 gato(chanel).
7 gato(orion).
8 ave(piolin).
9
10 dueño(ana, firulaís).
11 dueño(ana, misu).
12 dueño(luis, luna).
13 dueño(luis, orion).
14 dueño(luis, firulaís).
15 dueño(maria, piolin).
16 dueño(julia, chanel).
17 dueño(pedro, bruno).
18
19
20 tiene_perro(Persona) :-
21     dueño(Persona, Mascota),
22     perro(Mascota).
23
24
25
26 dueño_perro(Persona, Mascota) :-

```

dueno_multiples_tipos(Persona).

Persona = ana
Persona = luis
Persona = luis

amante_animales(Persona).

Persona = ana
Persona = luis
Persona = luis

masкота_compartida(Persona1, Persona2).

Persona1 = ana,
Persona2 = luis
Persona1 = luis,
Persona2 = ana

tipo_mascota(Persona, Tipo).

Persona = ana

?- tipo_mascota(Persona, Tipo).

Examples History Solutions ☐ table results Run

File Edit Examples Help

Program

+

```

1 perro(firulaís).
2 perro(bruno).
3 perro(max).
4 gato(misu).
5 gato(luna).
6 gato(chanel).
7 gato(orion).
8 ave(piolin).
9
10 dueno(ana, firulaís).
11 dueno(ana, misu).
12 dueno(luis, luna).
13 dueno(luis, orion).
14 dueno(luis, firulaís).
15 dueno(maria, piolin).
16 dueno(julia, chanel).
17 dueno(pedro, bruno).
18
19
20 tiene_perro(Persona) :-
21     dueno(Persona, Mascota),
22     perro(Mascota).
23
24
25

```

Persona2 = ana

tipo_mascota(Persona, Tipo)

Persona = ana,

Tipo = perro

Persona = luis,

Tipo = perro

Persona = pedro,

Tipo = perro

Persona = ana,

Tipo = gato

Persona = luis,

Tipo = gato

Persona = luis,

Tipo = gato

Persona = julia,

Tipo = gato

Persona = maria,

Tipo = ave

?- tipo_mascota(Persona, Tipo).

Examples

History

Solutions

☐ table results

Run

File Edit Examples Help

Program

+

```

39 Nota >= 4.0,
40 Nota < 4.5.
41
42 clasificacion(Estudiante, excelente) :-
43     nota(Estudiante, Nota),
44     Nota >= 4.5,
45     Nota <= 5.0.
46
47
48
49
50 distancia((X1, Y1), (X2, Y2), D) :-
51     D is sqrt((X2-X1)^2 + (Y2-Y1)^2).
52
53
54
55 distancia_total([], 0).
56
57 distancia_total([_], 0).
58
59
60
61 distancia_total([P1, P2 | Resto], Total) :-
62     distancia(P1, P2, D),
63     distancia_total([P2 | Resto], TotalResto),
64     Total is D + TotalResto.

```

reprueba(Estudiante)

Estudiante = luis

Estudiante = pedro

aprueba(Estudiante)

Estudiante = ana

Estudiante = maria

Estudiante = juan

rango(Estudiante, 3.0, 4.0)

Estudiante = maria

false

?- rango(Estudiante, 3.0, 4.0).

Examples

History

Solutions

☐ table results

Run

Ejercicio4

File Edit Examples Help

Program

+

```

1 fibonacci(0, 0).
2 fibonacci(1, 1).
3
4 fibonacci(N, F) :-
5     N > 1,
6     N1 is N - 1,
7     N2 is N - 2,
8     fibonacci(N1, F1),
9     fibonacci(N2, F2),
10    F is F1 + F2.

```

fibonacci(0, X).

X = 0

false

fibonacci(1, X).

X = 1

false

fibonacci(5, X).

X = 5

false

fibonacci(7, X).

X = 13

false

?- fibonacci(7, X).

Examples

History

Solutions

☐ table results

Run

Ejercicio5

The image shows the SWISH web interface for a Prolog program. The interface includes a menu bar with 'File', 'Edit', 'Examples', and 'Help'. A search bar is located in the top right corner. The main area is divided into two panes. The left pane, titled 'Program', contains the following Prolog code:

```
1 color(red).
2 color(blue).
3 color(yellow).
4 color(green).
5 adjacent(A, B) :-
6     A \= B.
7 map(A, B, C, D, E) :-
8     color(A),
9     color(B),
10    color(C),
11    color(D),
12    color(E),
13
14    adjacent(A, B),
15    adjacent(A, D),
16    adjacent(A, E),
17
18    adjacent(B, C),
19    adjacent(B, D),
20    adjacent(B, E),
21
22    adjacent(C, D),
23    adjacent(C, E),
24
25    adjacent(D, E).
```

The right pane, titled 'map(A, B, C, D, E).', displays the execution results. The results are organized into three sections, each starting with a question mark '?' and followed by a 'map(A, B, C, D, E).' query. The first section shows a solution where A=red, B=blue, C=yellow, D=green, and E=green. The second section shows a solution where A=red, B=blue, C=yellow, D=blue, and E=green. The third section shows a solution where A=red, B=blue, C=yellow, D=green, and E=blue. At the bottom of the right pane, there are tabs for 'Examples', 'History', and 'Solutions', and a 'Run' button.

Ejercicio6

The image displays two screenshots of the SWISH Prolog environment, showing code execution and results.

Top Screenshot:

- Code:**

```
1 miembro(X, [X|_]).
2 miembro(X, [_|T]) :-
3   miembro(X, T).
4
5
6
7 longitud([], 0).
8
9 longitud([_|T], N) :-
10  longitud(T, N1),
11  N is N1 + 1.
12
13
14
15 concatena([], L, L).
16
17 concatena([H|T], L2, [H|R]) :-
18  concatena(T, L2, R).
19
20
21 fibonacci(0, 0).
22 fibonacci(1, 1).
23
24 fibonacci(N, F) :-
25  N > 1,
26  N1 is N - 1,
```
- Results:**
 - Query: `miembro(3, [1,2,3,4]).` Result: `true`
 - Query: `miembro(7, [1,2,3,4]).` Result: `false`
 - Query: `?- miembro(7, [1,2,3,4]).` Result: `miembro(7, [1,2,3,4])`

Bottom Screenshot:

- Code:**

```
24 fibonacci(N, F) :-
25  N > 1,
26  N1 is N - 1,
27  N2 is N - 2,
28  fibonacci(N1, F1),
29  fibonacci(N2, F2),
30  F is F1 + F2.
31
32
33 fibonacci_lista(0, []).
34
35 fibonacci_lista(N, L) :-
36  N > 0,
37  N1 is N - 1,
38  fibonacci(N1, F),
39  fibonacci_lista(N1, L1),
40  append(L1, [F], L).
41
42
43 reverso([], []).
44
45 reverso([H|T], R) :-
46  reverso(T, R1),
47  append(R1, [H], R).
48
49
```
- Results:**
 - Query: `miembro(3, [1,2,3,4]).` Result: `true`
 - Query: `miembro(7, [1,2,3,4]).` Result: `false`
 - Query: `?- miembro(7, [1,2,3,4]).` Result: `miembro(7, [1,2,3,4])`